

Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Parts:

- Part 1: Complete 1 of 3 questions
- Part 2: Review your partner's Assignment 1 submission
- Part 3: Perform code review of your partner's assignment 1 by answering the questions below
- Part 3: Reflect on Assignment 1 and Assignment 2

Part 1:

You will be assigned one of three problems based of your first name. Enter your first name, in all lower case, execute the code below, and that will tell you your assigned problem. Include the output as part of your submission (do not clear the output). The problems are based-off problems from Leetcode.

```
In [ ]: import hashlib

def hash_to_range(input_string: str) -> int:
    hash_object = hashlib.sha256(input_string.encode())
    hash_int = int(hash_object.hexdigest(), 16)
    return (hash_int % 3) + 1
input_string = "your_first_name_here"
result = hash_to_range(input_string)
print(result)
```

► Question 1

Starter Code for Question 1

```
In [ ]: # Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
def is_duplicate(root: TreeNode) -> int:
    # TODO
```

► Question 2

Starter Code for Question 2

```
In [ ]: # Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
def bt_path(root: TreeNode) -> List[List[int]]:
    # TODO
```

► Question 3

Starter Code for Question 3

```
In [99]: def missing_num(nums: list) -> int:

    # sort values in the list
    nums.sort()

    # get unique values
    nums_set = set(nums)
    nums = list(nums_set)
    n = len(nums)
    print(f'Unique list: {nums_set}')

    # create a list to store missing values
    missing = []

    for i in range(1,n):
        # if value not missing within the range, append to 'missing' list
        if nums[i] - nums[i-1] > 1:
            for j in range(nums[i-1]+1, nums[i]):
                missing.append(j)

    if missing:
        return missing
    # if all values within range are present, return -1
    else:
        return -1

# nums = [5, 2, 4, 6, 1]
```

```
nums = [6,8,2,3,5,7,1,1,2,10]
missing_num(nums)
```

Unique list: {1, 2, 3, 5, 6, 7, 8, 10}

Out[99]: [4, 9]

In []:

```
# for i in range(1,n):
#     # if value not missing within the range, append to 'missing' list
#     if nums[i] - nums[i-1] != 1:
#         add_value = nums[i]-1
#         missing.append(add_value)
#     return missing
```

Part 2:

You and your partner must share each other's Assignment 1 submission.

Part 3:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

The problem asks to return the first value that is a duplicate in the given list. if there are mutiple duplicate, return the first occurrence and return -1 if there are no duplicates.

- Create 1 new example that demonstrates you understand the problem.
Trace/walkthrough 1 example that your partner made and explain it.

In []:

```
# Example 1
input1 = [0,9,0,6,5,4]
output1 = [0]

input2 = [3,4,7,8,4,7,8]
output2 = [4]
```

- Copy the solution your partner wrote.

In [80]:

```
from typing import List

def first_duplicate(nums: List[int]) -> int:
    seen = set()
```

```
for item in nums:
    if item in seen:
        return item
    else:
        seen.add(item)
return -1
```

```
In [81]: # nums = [3,4,7,8,4,7,8]
num = [0,1,2,3,8,9,7,0,3,2]
first_duplicate(nums)
```

```
Out[81]: 4
```

- Explain why their solution works in your own words.

José's solution works because he starts off creating an empty set, then for each value in the original list, if it is in the set, the value is returned. Otherwise, the value is added to the set. As the list is iterated through, the moment that it is seen again (already in the set), the "if" part of the statement is executed and the loop is broken, as it returns the first duplicate value.

- Explain the problem's time and space complexity in your own words.

Since there is a loop used to iterate through the list, the time complexity is $O(n)$ and the space complexity is $O(n)$ because a list is used to store the data.

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

I think José created an algorithm with optimal time complexity. Space complexity can't be improved as we were given the list to work with as the data structure.

Part 4:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

In assignment 1, I answered question 3 "Move All Zeros to the End". I used a bubble sort method (using a nested for loop) to reach the goal of the problem: move the zeros to the end of the list while maintaining the order of the other values. The first loop iterates through the length of the list. The second loop switches the non-zero integers with the zeros, ultimately moving the zeros to the end of the list. If the current value being evaluated is equal to zero and the next value in the list is not equal to zero, the values switch positions. The counter is incremented by one to continue the iteration.

My experience with my partner was straight forward. José was assigned the problem "First Duplicate in List". He explained his algorithm's function, time and space complexities very well. There was also a very clear explanation provided for an alternative method to solve the question he was assigned.

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

🔔 **Please review our [Assignment Submission Guide](#)** 🔔 for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: HH:MM AM/PM - DD/MM/YYYY
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
 - This Jupyter Notebook (assignment_2.ipynb) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment:
`https://github.com/<your\_github\_username>/algorithms\_and\_data\_structures/`

- Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- ☐ Created a branch with the correct naming convention.
- ☐ Ensured that the repository is public.
- ☐ Reviewed the PR description guidelines and adhered to them.
- ☐ Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at [#cohort-6-help](#). Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.

In []: